More Hooks

Ruby provides a motley bunch of hooks that cover the most newsworthy events in the object model. Just as you override Class#inherited to plug into the lifecycle of classes, you can plug into the lifecycle of modules by overriding Module#included and (in Ruby 2.0) Module#prepended:

```
module M1
  def self.included(othermod)
    puts "M1 was included into #{othermod}"
  end
end

module M2
  def self.prepended(othermod)
    puts "M2 was prepended to #{othermod}"
  end
end

class C
  include M1
  prepend M2
end
```

```
M1 was included into C
M2 was prepended to C
```

You can also execute code when a module extends an object by overriding Module#extended. Finally, you can execute method-related events by overriding Module#method_added, method_removed, or method_undefined.

```
module M
  def self.method_added(method)
    puts "New method: M##{method}"
  end

  def my_method; end
end
```

```
New method: M#my_method
```

These hooks only work for regular instance methods, which live in the object's class. They don't work for *Singleton Methods* (*114*), which live in the object's singleton class. To catch Singleton Method events, you can use BasicObject#singleton_method_added, singleton_method_removed, and singleton_method_undefined.

Module#included is probably the most widely used hook, thanks to a common metaprogramming spell that's worthy of an example of its own.